

SRN: PES1PG25CA044

Name: Avi Chovatiya

MVC Pattern: The Foundation of Modern Web Development

Object Oriented Software Engineering Power-Point Presentation

Contents To Be Covered

- 01 Description
- 02 Need for MVC
- 03 The Problem
- 04 Its Solution
- 05 Real Time Example
- 06 Real Time Example

Model

Data and Updates

Handles the data and business logic of the Application

- Manages and Stores Data
- Enforces business rules
- Responds to the requests from the controller

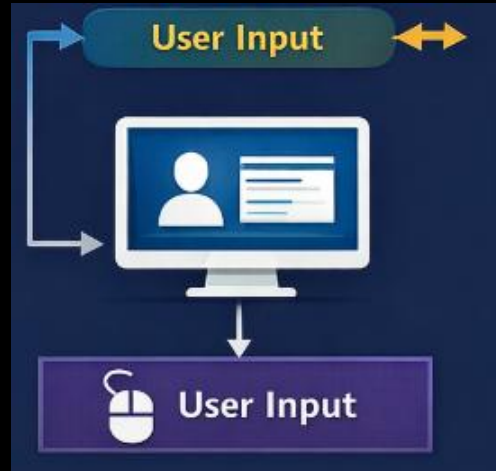


View

Displays the data and interacts with the user

Renderes the User Interface and presents the data to the user

Recieves input from the user and sends it to the controller

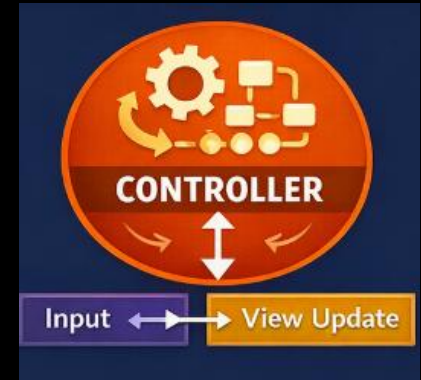


Controller

Acts as an intermediary between the Model and the View, controlling the flow of data

- Processes user input
- Updates the model
- Refreshes the view

Receives input, interacts with the model, and updates the view



Need for MVC

MVC is a widely used architecture pattern in software development. It splits an application into three separate components, so the code stays organized and doesn't become messy or confusing.

The main concept behind MVC is **Separation of Concerns**, which simply means each part has its own responsibility and should focus only on its own work without interfering with the others.

THE Problem

* **Monolithic Code**

- Everything is written in one big piece of code.
- If you change one small line, the whole app can stop working.

* **UI and Logic Mixed Together**

- The screen design and the main working logic are stuck together.
- If you change the UI, the logic may get affected too.

* **Hard to Test and Maintain**

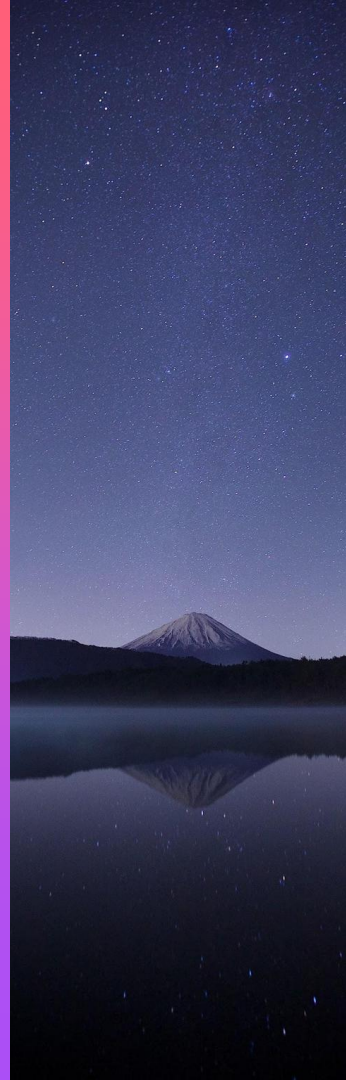
- Testing becomes very difficult and confusing.
- Fixing errors takes more time than writing the program.

* **Code Duplication**

- The same code is repeated in many places.
- It feels like the code is always being copy-pasted.

* **UI Changes Break Logic**

- Even a small UI change (like changing a button color) can break the system.
- That's a sign the code is not properly organized.



THE Solution



MVC separates **data (Model)**, **interface (View)**, and **request handling (Controller)**, following the flow **User → Controller → Model → View → User**, which makes the application easier to **maintain, scale, and test**.

Real-Time Example of MVC

- Model: Kitchen + ingredients (stores data and prepares food)
- View: Menu + served dish (what the customer sees)
- Controller: Waiter (takes the order and sends it to the kitchen)

Flow: Customer → Waiter → Kitchen → Dish served → Customer 

Conclusion

In conclusion, **MVC (Model-View-Controller)** is like a well-organized team where each part has its own responsibility. The **Model** handles all the important work like storing data and applying business rules (the “brain” of the application). The **View** is the part the user sees, such as pages, buttons, text, and the overall design (the “face” of the application). The **Controller** works as the manager—it takes the user’s actions (like clicking a button), decides what should happen next, updates the Model if needed, and then tells the View what to display.

THANK YOU